Speedrunning Social Development: How We Can Improve How We Improve

Tyler Roost

Abstract

Digital platforms shape an increasing share of human social development: how people communicate, learn, coordinate, create, belong, and build identity. Yet most platforms are still optimized through blunt proxies such as engagement, retention, conversion, and monetization. Those metrics can be useful, but they do not reliably tell a platform how to become better at becoming better. They help a system extract value. They do not necessarily help it learn.

This paper presents a **paradigm-shifting conceptual framework** for the meta-improvement of digital products and digital social infrastructure. The core claim is that platforms should not only optimize user-facing experiences; they should also optimize the mechanisms by which they discover failure, absorb criticism, reward valuable contributors, and redesign their own metrics and ranking systems. The framework combines a user-satisfaction hyper-objective, structured rating-plus-feedback loops, layered contributor incentives, leaderboard critique, glitch-hunting subcultures, and a four-part hypertopology of learning: **Human-Compliant, Human-Adversarial, Tool-Compliant, and Tool-Adversarial**.

The comparison point is speedrunning. Long-lived speedrunning communities are resilient because they know how to metabolize breakage. A new exploit does not automatically destroy the scene. It often creates new categories, new routes, new verification norms, and deeper collective understanding. This paper argues that digital platforms should do the same. For builders, this means more robust product learning. For project managers, it means a cleaner loop from user signal to roadmap decisions. For investors, it means a stronger theory of compounding product quality and better investment judgment through lower dependence on internal fiction and proxy blindness.

1 Introduction

The digital systems through which people live increasingly function as developmental environments. A social network, chatbot, game, productivity tool, streaming platform, or communication surface is not just software. It is part of the social environment in which people form habits, relationships, identities, skills, expectations, and beliefs about what is possible.

If these environments are optimized badly, millions of people experience shallow, manipulative, or confusing systems as normal. If they are optimized well, the quality of social life can improve at scale.

The usual explanation for digital decay is human nature: people are status-seeking, distractible, tribal, impulsive, and easy to manipulate. This explanation contains some truth but is too incomplete to be useful as a design philosophy. The deeper issue is that most platforms do not actually know how to learn from their own failure modes beyond a narrow set of business proxies. They are

optimized to survive, grow, and monetize, but not to become increasingly wise about what their users value, how their incentive systems distort behavior, or how their own internal metrics can be gamed.

This paper proposes a different design frame: a platform should be treated as a system that must learn **how to improve how it improves**. It is offered as a paradigm-shifting conceptual framework rather than as a conventional empirical paper. Its goal is not to bury the reader in exhaustive measurement before the design space is visible. Its goal is to formalize a stronger design target first, then make empirical testing easier to structure later.

That shift matters at every layer of a company.

For builders, it means product architecture should support not just features, but learning loops.

For project managers, it means the roadmap should be informed not only by raw feedback or analytics, but by increasingly structured, contestable, and high-value feedback. It also reduces **internal fiction**: internal metrics, narratives, or coordination stories that help the company cohere but do not reliably track user-valued improvement.

For investors, it means product quality can be understood as a compounding capability rather than a static artifact. The strongest products are not just good. They are good at learning why they are not good enough yet.

2 The Intended Value: User-Satisfaction

Every platform implicitly optimizes something. The first design question is therefore simple: what should the platform treat as its intended value?

This paper proposes that the right hyper-objective is **user-satisfaction**, carefully defined. This choice is adjacent to long-running work in usability, user experience, experimentation, and optimization, but it is not identical to any one of those traditions. It is a proposal about what the platform should ultimately be trying to learn, not merely what it should measure most often.

Not shallow pleasantness. Not time spent. Not addictive stickiness. Not investor confidence. Not even revenue by itself.

Rather:

> **User-satisfaction is the degree to which a user, after sufficiently understanding and using the platform for their own goals, would voluntarily endorse continued use with low regret, low felt deficiency, and meaningful belief that the platform serves them well relative to realistic alternatives.**

This is the right starting point for three reasons.

First, it is closer to the actual purpose of a product than downstream financial abstractions. Revenue measures captured value. User-satisfaction is a stronger indicator that value was provided.

Second, it is upstream of many desirable business outcomes: stronger retention, lower churn, better word of mouth, more pricing power, better contributor quality, and more stable growth.

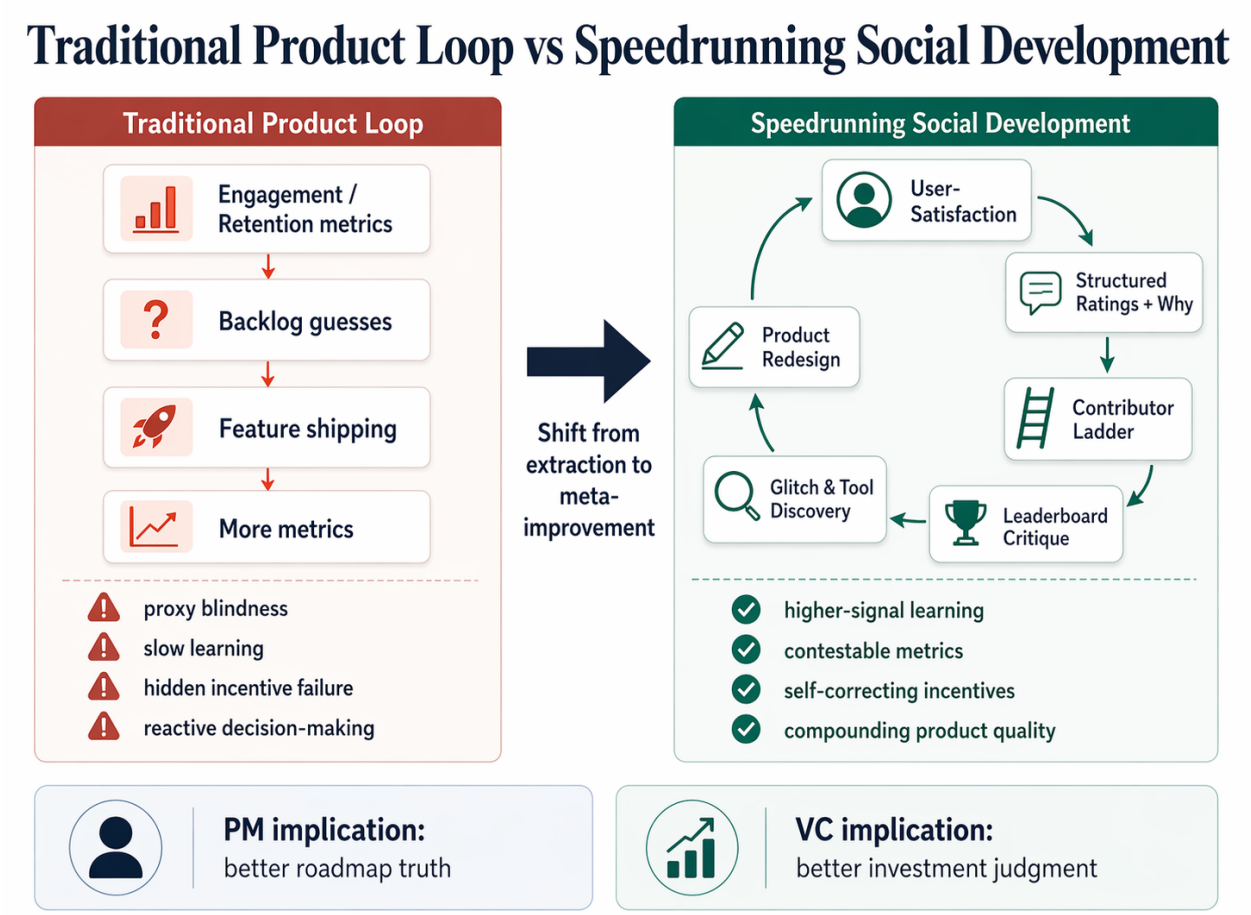

Figure 1: Traditional product loops optimize within blunt proxy systems; Speedrunning Social Development adds structured user signal, contributor development, leaderboard critique, and glitch/tool discovery to produce a higher-signal, self-correcting learning system.

Third, it creates a cleaner moral and operational boundary against some of the worst platform pathologies. Engagement can rise through compulsion or outrage. Retention can rise through switching costs. Monetization can rise through extraction. Deep user-satisfaction is harder to fake durably.

This does not mean user-satisfaction should be treated as a perfect single scalar. It should be treated as the intended value and inferred through multiple observables. The most important user-facing observable is a clearly defined five-star system.

2.1 A Five-Star System With Real Semantics

- **1 star** - I regret using this app. It failed my needs or made things worse.
- **2 stars** - There is some value here, but it is frustrating enough that I would avoid using it if I had a reasonable alternative.

- **3 stars** - It is useful and usable, but clearly incomplete or flawed for my needs.
- **4 stars** - This app works well for me. I would willingly keep using it, and only minor improvements come to mind.
- **5 stars** - I literally cannot imagine how this app could be better for what I need right now.

The purpose of this scale is not merely to gather sentiment. It is to help users locate their own experience in a legible way and to make that experience more actionable for the platform.

3 The First Flywheel: Hyper-Objective → User → Feedback → Hyper-Objective

A rating alone is too compressed. A freeform feedback box alone is too unconstrained. Put together, they create a usable first loop.

After the user selects a rating, the platform asks:

> **Why did you provide this rating: [clear star description]?**

This matters because it converts a bounded self-location into causal material. The user is no longer just reacting. They are beginning to articulate why the platform feels the way it does.

This creates the first flywheel:

> **hyper-objective → user → feedback → hyper-objective**

But most platforms stop here. They gather signal, perhaps label it, perhaps route it into a backlog, and then treat the loop as complete. It is not complete. A platform that wants to improve how it improves must also learn how to identify high-value contributors, how to reward them, how to train them, and how to redesign its own feedback and ranking systems when those systems become brittle.

4 Contributor Development and Incentive Design

A major failure of existing feedback systems is that they extract value from users without acknowledging that some of those users are doing real product-improvement work.

If a user materially improves the platform, they should not be treated as free emotional exhaust. They should be treated as contributors.

4.1 The Contributor Ladder

Layer 1 - Standard users
Occasional ratings and optional feedback.

Layer 2 - Access dividend plus reflective training
If a user's feedback creates meaningful value, the platform grants an access dividend such as a free month or quarter. Reflective training should co-occur with this reward, not precede it. The user should experience the platform as saying: you already created value, we are returning some of it, and we are also helping you increase your future impact.

Layer 3 - Mixed compensation
At higher levels of contribution, the platform must explicitly state that cash is part of the upward path. Platform benefits alone are not enough. Internal rewards are good continuity signals; cash is the reality signal.

Layer 4 - Structured contribution contracts
At this stage, contributors may commit to bounded periods of work around defined matters: onboarding failures, tutorial confusion, feature questions, leaderboard issues, exploit discovery, or satisfaction bottlenecks.

Layer 5 - Reputation, community, and status
Layer 5 is a contributor-status environment visible only to serious contributors. It allows competition, idea sharing, firstness recognition, and visible status among users who materially improve the platform.

4.2 Hyper-Feedback Providers

Some users are not merely good feedback users. They are **hyper-feedback providers**. They surface system-level issues, identify hidden assumptions, distinguish tutorial failure from feature failure, point out metric brittleness, and often improve the feedback system itself.

These users should be treated as a distinct contributor class. More importantly, the platform should not assume that such contributors either exist in finished form or cannot be cultivated. One of the platform's meta-learning tasks is to develop more of them.

For builders, this creates a richer source of product intelligence.

For PMs, it creates a clearer gradient from raw user noise to high-quality product signal, while reducing the gap between dashboard belief and lived user reality.

For investors, it creates a nontrivial moat: a product that gets better at generating and developing high-value contributors can improve faster and with less internal fiction than competitors.

5 Leaderboards as Governed Objects

The moment a contributor class exists, the platform has another design problem: how to rank or recognize contributors without collapsing into a bad status game.

The answer is not to avoid leaderboards entirely. The answer is to treat leaderboards as **governed objects**.

A serious contributor should be able to say:

> I have a problem with the leaderboard. It fails because of this. I have noticed it explicitly

here and implicitly here.

This creates the second flywheel:

> **leaderboard → contributor reaction → leaderboard critique → leaderboard redesign → better leaderboard**

A single-number throne is too weak to describe ongoing contributor value. But ceremonial firstness still matters. Threshold-crossing events deserve memory. The first person to surface a new exploit class or a new contributor role deserves a kind of recognition that should not be flattened into a rolling score.

6 Goodhart, Glitches, and Why Speedrunning Matters

Every proxy system invites gaming. Multiple orthogonal KPIs help, but they do not solve the problem completely. A sufficiently clever optimizer can still satisfy a bundle of metrics while drifting away from the intended value. This family of failure modes is well described by adjacent work on proxy optimization, target distortion, and institutional measurement pathologies.

This is where the speedrunning analogy becomes unusually useful.

Long-running speedrunning communities survive because they know how to metabolize breakage. A newly discovered exploit does not automatically destroy the scene. It often produces:

- new categories
- new routing strategies
- new norms of verification
- new distinctions between lawful and exploitative runs
- new prestige for discovery, not only execution

That is what makes the analogy so powerful for platform design. A healthy system is not one that never breaks. It is one that can absorb breakage and become more structurally intelligent because of it.

This is relevant to builders because it changes how edge cases are treated. It is relevant to PMs because it changes how failure reports become product insight rather than support burden. It is relevant to investors because it reframes resilience: the best products are not just hard to break; they are good at learning from how they break.

7 The Final Missing Piece: A Four-Part Hypertopology of Learning

The platform should distinguish two axes:

- **Compliant vs Adversarial**
- **Human vs Tool**

This yields four learning regimes.

7.1 Human-Compliant

Ordinary lawful human use. This reveals real satisfaction, real friction, realistic onboarding performance, and the lived experience of the intended product.

7.2 Human-Adversarial

Human glitch-hunting. This reveals loopholes, prompt failures, ranking exploits, and incentive distortions that real people can actually reach.

7.3 Tool-Compliant

Tool-assisted lawful search. This reveals best-known compliant strategies, lawful upper bounds, and where current human usage sits far below the product's legitimate frontier.

7.4 Tool-Adversarial

Tool-assisted exploit discovery. This reveals latent exploit classes, proxy-breaks, hidden rule interactions, and failure surfaces humans may never naturally discover even though the system technically permits them.

7.5 Why this is not merely a matrix

The value is not only in the four boxes. It is in the transitions among them.

Tool-Adversarial can find an exploit that Human-Adversarial later reproduces. Human-Adversarial can force a redesign that improves Human-Compliant experience. Tool-Compliant can discover a lawful optimum that later informs better tutorials, defaults, or workflows.

The platform therefore learns not only from states, but from paths across those states. That is why this is a **hypertopology of learning**, not merely a 2x2 diagram.

For builders, this gives a fuller search surface.

For PMs, it clarifies which kinds of findings should affect roadmap, policy, tutorials, ranking, or incentive design.

For investors, it describes a company that is not merely collecting analytics but building a higher-order capability for product discovery and self-correction.

8 Why This Matters Across the Stack

This framework is not only a philosophical proposal. It has different practical meanings for different layers of product and capital formation.

8.1 For builders

Build systems that do not just ship features. Build systems that learn from failure, criticism, adversarial discovery, and tool-assisted search.

8.2 For project managers

Do not reduce product intelligence to analytics dashboards and unstructured user comments. Build a path from user-satisfaction to structured feedback to contributor development to roadmap quality.

8.3 For venture capitalists and capital allocators

A company with a stronger meta-improvement loop is not merely a company with a better product today. It is a company with a better mechanism for finding tomorrow's product truth faster, with less internal fiction and less proxy blindness. That should matter to diligence and ultimately to investment judgment.

9 Positioning and Adjacent Literature

This paper is intentionally framework-first. It does not claim that the design space described here already has a complete empirical literature under one shared name. Rather, it argues that multiple adjacent literatures point toward the need for a stronger synthesis.

The critique of proxy optimization and target distortion is adjacent to Goodhart's and Campbell's law. The emphasis on experimentation and structured learning is adjacent to work on online controlled experiments and business experimentation. The user-satisfaction framing is adjacent to usability and user experience research, but it is here elevated from a measurement concern to an intended-value concern. The Human/Tool x Compliant/Adversarial hypertopology is adjacent to adversarial testing, red teaming, reward learning, and the broader problem of aligning system optimization with what humans actually value. The speedrunning comparison is adjacent to broader work on how rule systems, player communities, exploits, and interpretation co-evolve over time.

The contribution claimed here is therefore not that every component is unprecedented in isolation. The contribution is a new synthesis: a paradigm-shifting conceptual framework for building digital systems that improve how they improve.

10 Conclusion

Digital infrastructure is not failing only because humans are flawed. It is failing because most platforms were not designed to become wiser about their own failure modes.

The next generation of products should not merely optimize features. They should optimize the systems by which features are criticized, repaired, re-ranked, retaught, and re-understood. They should treat user-satisfaction as the intended value, cultivate contributors instead of extracting from them, make ranking systems contestable, and build formal pathways for glitch-hunting and tool-assisted discovery.

The deepest lesson from speedrunning is not speed. It is social resilience under discovery. A healthy community does not fall apart when someone finds a break in the rules. It becomes more sophisticated.

That is the core claim of this paper.

To improve human lives, we need better social experiences. To build better social experiences, we need better digital systems. And to build better digital systems, we need systems that improve how they improve.

References

1. Amodei, Dario, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. *Concrete Problems in AI Safety.* arXiv:1606.06565.
2. Campbell, Donald T. 1976. *Assessing the Impact of Planned Social Change.* Hanover, NH: The Public Affairs Center, Dartmouth College.
3. Christiano, Paul F., Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. *Deep Reinforcement Learning from Human Preferences.* In *Advances in Neural Information Processing Systems 30.*
4. Consalvo, Mia. 2007. *Cheating: Gaining Advantage in Videogames.* Cambridge, MA: MIT Press.
5. Goodhart, Charles A. E. 1975. *Problems of Monetary Management: The U.K. Experience.* In *Papers in Monetary Economics.* Sydney: Reserve Bank of Australia.
6. Hassenzahl, Marc, and Noam Tractinsky. 2006. *User Experience - A Research Agenda. Behaviour & Information Technology* 25(2): 91–97.
7. Kohavi, Ron, Diane Tang, and Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing.* Cambridge: Cambridge University Press.
8. Newman, James A. 2008. *Playing with Videogames.* London & New York: Routledge.
9. Nielsen, Jakob. 1994. *Usability Engineering.* San Francisco, CA: Morgan Kaufmann.
10. Salen, Katie, and Eric Zimmerman. 2003. *Rules of Play: Game Design Fundamentals.* Cambridge, MA: MIT Press.

11. Sutton, Rich. 2019. *The Bitter Lesson*. March 13, 2019. Available at http://www.incompleteideas.net/IncIdeas/BitterLesson.html.

12. Thomke, Stefan. 2020. *Experimentation Works: The Surprising Power of Business Experiments*. Boston, MA: Harvard Business Review Press.